AI-Native Transceiver Design for Next-Generation Wireless Networks

Yuxing Mao and Haikun Xu | Supervisor: Dr Huynh Nguyen

Trustworthy Computing, School of Computer Science and Informatics · LivSURF 436026

Notebook and measured data

1 Background and Motivation

A conventional transceiver is built as a chain of blocks, each designed on its own. The chain performs well, but only as well as the assumptions that link one block to the next. Deep learning suggests two ways around this: replace the whole chain with an **autoencoder** [1], or replace channel estimation and detection with a **learned receiver** [2].

Two objections are usually raised, and both are fair:

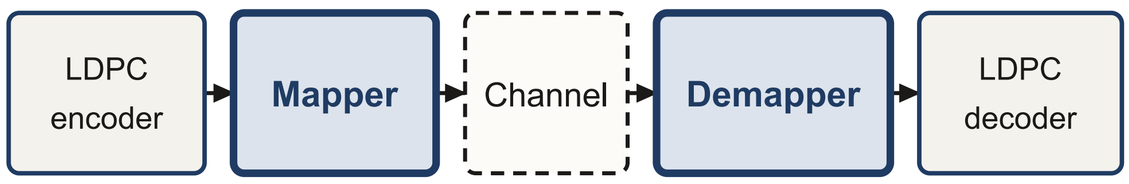
- Training needs a **differentiable model of the channel**. A real channel is a black box, so this cannot be done on deployed hardware.
- Most published gains are measured on **AWGN**, where the classical receiver is already optimal. A gain measured there may not survive on a realistic channel.

We address both by measurement.

2 Objectives and Contributions

- Measure what a learned transceiver gains over a **matched** classical baseline, on AWGN and on Rayleigh block fading with **pilot-estimated CSI**.
- Train the transmitter with **no channel model at all**, using reinforcement learning in place of backpropagation through the channel, and measure what this costs.
- Account for everything the learned system uses. The transmitter never sees the channel. Pilots are charged against the code rate as **R(L−P)/L**, and estimation error enters the effective noise as $\mathbf{N_0(1+1/P)/|\hat{h}|^2}$. Leaving out that last factor understates the noise by **1.76 dB** at P = 2.
- Explain each failure by **intervention**: hold the weights fixed, change one thing, and see which change removes the symptom.

3 System Model

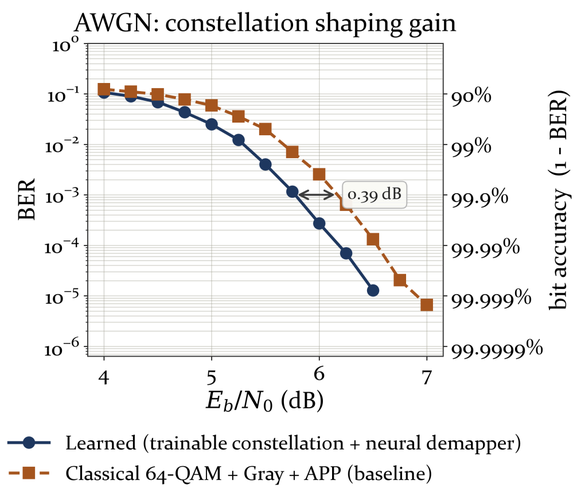


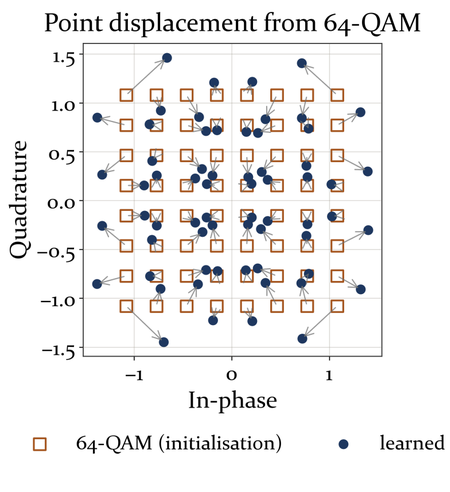
Learned	trainable constellation	neural network
Classical	64-QAM, Gray	APP demapper

Channel: AWGN | Rayleigh block fading, L = 25, P pilot symbols, LS estimate

The two systems differ only in the shaded blocks. They share the rate-1/2 5G LDPC code, the channel model and the random seeds, so any difference in bit error rate comes from the mapper and the demapper.

4 Where the AWGN Gain Comes From





Under AWGN the APP demapper is already optimal, so the receiver cannot account for the gain. It comes from the transmitter instead. The learned constellation moves off the regular lattice and gives more power to the symbols that were easiest to confuse.

5 Performance Analysis

Table 1 — E_b/N_0 needed for a given bit error rate (dB, lower is better). 64,000 codewords per point; every entry rests on at least 30 block errors.

System	1e-2	1e-3	1e-4
AWGN, classical 64-QAM + APP	5.67	6.17	6.54
AWGN, learned (channel gradient)	**5.30**	**5.78**	**6.18**
AWGN, learned (RL, no channel model)	5.42	5.92	6.31
Rayleigh, classical 64-QAM + APP	11.26	13.30	14.79
Rayleigh, learned	11.25	13.19	14.94

- Fading costs between **5.9 and 8.8 dB**, depending on the target error rate. Changing the receiver moves the curve by less than **0.4 dB**, an order of magnitude less.
- On AWGN the learned system wins by **0.37, 0.39 and 0.35 dB** at BER 1e-2, 1e-3 and 1e-4. The APP demapper is already optimal there, so this is **constellation shaping alone**.
- Under Rayleigh fading with estimated CSI we found **no measurable gain** (+0.01, +0.11 and −0.15 dB). The value at 1e-4 changed sign between two runs of the same systems, so we treat it as noise.

6 Why AWGN Weights Fail on Fading

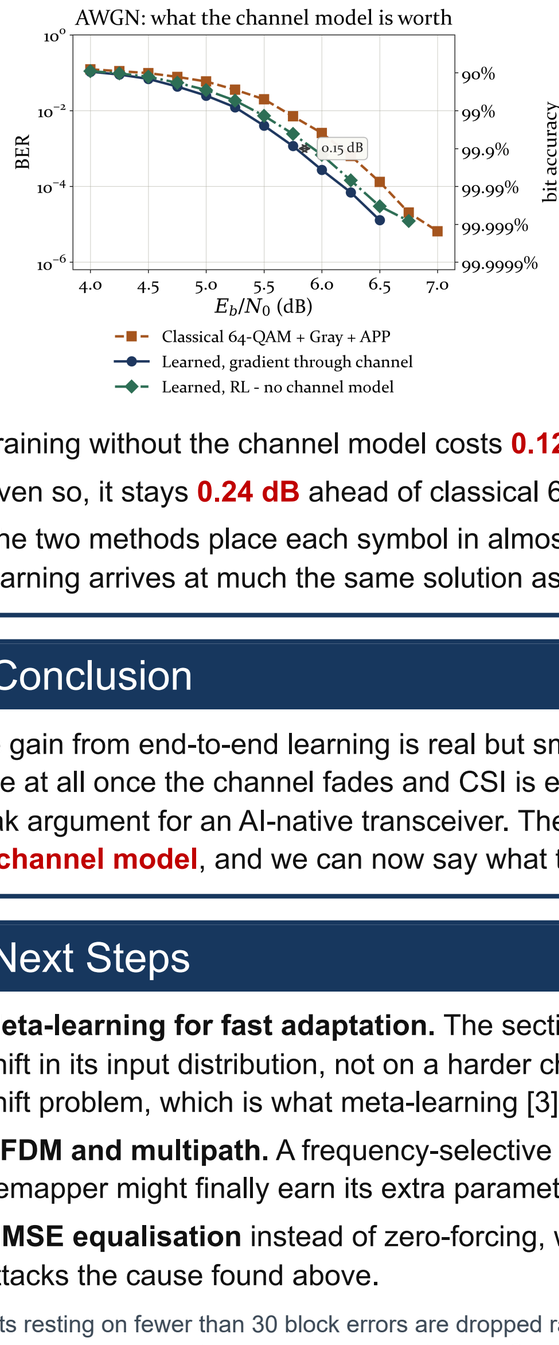
If the AWGN weights are reused on a fading channel without retraining, the BER curve rises with SNR. That cannot be physical, so we held every weight fixed and changed one thing at a time:

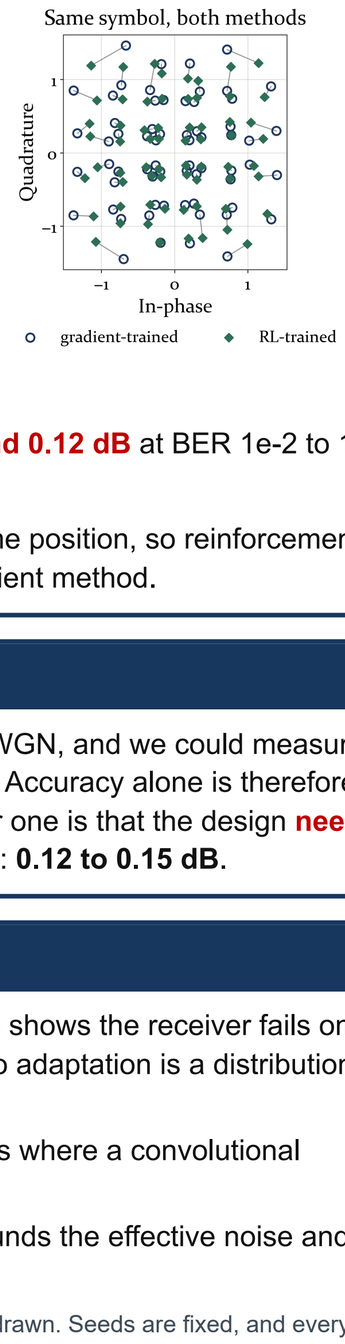
- Clamping the network outputs changed the result by under 4%, and the curve still rose.
- Erasing the symbols in deep fades made matters worse above 10 dB.
- Clamping the noise reported to the demapper back into its training range **made the curve fall properly again**, with no weight changed.

The problem is on the input side, and in the quiet tail rather than the deep fades: at 18 dB, **89%** of symbols arrive with an effective noise lower than anything seen in training, against 4.6% in deep fades. Retraining fixes it, reaching **1e-3** at 13.2 dB and **1e-4** at 14.9 dB.

7 Learning Without a Channel Model

Everything above was trained by backpropagating through a differentiable channel, which a deployed transmitter does not have. Here the receiver trains as usual, but its gradient never reaches the channel. The transmitter perturbs its own symbols, gets back a single scalar per codeword, and updates from that alone [4], so the channel is only ever sampled.





- Training without the channel model costs **0.12, 0.15 and 0.12 dB** at BER 1e-2 to 1e-4.
- Even so, it stays **0.24 dB** ahead of classical 64-QAM.
- The two methods place each symbol in almost the same position, so reinforcement learning arrives at much the same solution as the gradient method.

8 Conclusion

The gain from end-to-end learning is real but small on AWGN, and we could measure none at all once the channel fades and CSI is estimated. Accuracy alone is therefore a weak argument for an AI-native transceiver. The stronger one is that the design **needs no channel model**, and we can now say what that costs: **0.12 to 0.15 dB**.

9 Next Steps

- **Meta-learning for fast adaptation.** The section above shows the receiver fails on a shift in its input distribution, not on a harder channel, so adaptation is a distribution-shift problem, which is what meta-learning [3] is for.
- **OFDM and multipath.** A frequency-selective channel is where a convolutional demapper might finally earn its extra parameters.
- **MMSE equalisation** instead of zero-forcing, which bounds the effective noise and so attacks the cause found above.

Points resting on fewer than 30 block errors are dropped rather than drawn. Seeds are fixed, and every number is measured in the accompanying notebook. Sionna [5] and PyTorch. **Scope:** flat block fading only, with no multipath, Doppler or OFDM.

References

[1] T. O'Shea and J. Hoydis, "An Introduction to Deep Learning for the Physical Layer," arXiv:1702.00832, 2017.

[2] H. Ye, G. Y. Li and B.-H. F. Juang, "Power of Deep Learning for Channel Estimation and Signal Detection in OFDM," arXiv:1708.08514, 2017.

[3] S. Smeka J. et al., "Accelerating Meta-Learning with the Enhanced Reptile Algorithm for Rapid Adaptation in Neural Networks," SCSE, IEEE, 2025.

[4] N. C. Luong et al., "Applications of Deep Reinforcement Learning in Communications and Networking: A Survey," IEEE Commun. Surveys Tuts., vol. 21, no. 4, 2019.

[5] NVIDIA Sionna, autoencoder tutorial notebook, which this work extends.